# BACHELOR OF COMPUTER APPLICATIONS

## SEMESTER 1

# INTRODUCTION TO WEB PROGRAMMING

# Unit 7

# Introduction to JavaScript

## Table of Contents


| SL No | Topic | Fig No / Table / Graph | SAQ / Activity | Page No |
|---|---|---|---|---|
| 1 | Introduction | - | - | 3 |
| | 1.1 Learning Objectives | - | - | |
| 2 | The JavaScript Language | - | - | 4-10 |
| 3 | History and Versions | - | - | 11-14 |
| 4 | Syntax | - | - | 15-17 |
| 5 | Variables and Datatypes | - | - | 18-21 |
| 6 | Statements | - | - | 21-22 |
| 7 | Operators | - | - | 23-27 |
| 8 | Summary | - | - | 28 |
| 9 | Questions | - | 1 | 29 |
| 10 | Answers | - | - | 30 |

# 1. INTRODUCTION

The chapter is a foundational exploration of one of the most crucial programming languages used in web development and beyond. JavaScript has come a long way since its inception, and this chapter delves into its history, evolution through various versions, and the core syntax that underlies its functionality. It covers fundamental concepts such as variables and data types, the structure of statements, and the powerful operators that enable developers to create dynamic and interactive web applications. Whether you're a beginner or a seasoned developer, understanding these fundamental aspects of JavaScript is essential for harnessing its full potential in modern web development.

## 1.1 Learning Objectives

*By the end of this chapter you will be able to:*

- *To recall and describe the historical development of JavaScript, including key milestones and versions, and explain its significance in the world of web development.*
- *Comprehend and summarize the fundamental syntax rules of JavaScript, distinguishing between correct and incorrect code structures, and explaining how syntax errors can impact program execution.*
- *Apply their understanding of JavaScript variables and data types to create and manipulate variables, demonstrating how to store and retrieve data effectively.*
- *Analyze JavaScript code snippets and identify the purpose and functionality of various statements and operators, including conditional statements, loops, and mathematical operators, to solve programming challenges.*
- *Synthesize their knowledge of JavaScript syntax, variables, statements, and operators to design and implement small JavaScript programs or scripts that perform specific tasks or add interactive features to web pages, showcasing their problem-solving and coding abilities.*

## 2. THE JAVASCRIPT LANGUAGE

JavaScript is a lightweight, cross-platform, single-threaded, and interpreted compiled programming language. It is also known as the scripting language for webpages. It is well-known for the development of web pages, and many non-browser environments also use it.

The provided passage offers an overview of JavaScript, highlighting its key characteristics and roles in programming.

1. *Lightweight and Cross-Platform:* JavaScript is considered lightweight because it doesn't require extensive system resources to run. It is also cross-platform, meaning it can work on various operating systems, making it highly versatile and accessible.
2. *Single-Threaded:* JavaScript is single-threaded, which means it executes one operation at a time, making it suitable for tasks that don't require multi-threading capabilities. This is particularly relevant in web development, where JavaScript is often used for tasks like user interface interactions.
3. *Interpreted and Compiled*: JavaScript is both an interpreted and compiled language. The source code is first compiled by the browser's JavaScript engine and then executed. This compilation process is generally fast, contributing to JavaScript's speed and efficiency.
4. *Scripting Language for Webpages:* JavaScript is widely known as the scripting language for webpages. It allows web developers to enhance the interactivity and functionality of web pages by adding client-side scripts that respond to user actions.
5. *Client-Side and Server-Side Development:* JavaScript is versatile and can be used for both client-side and server-side development. On the client-side, it enhances the user experience by enabling dynamic content and interactions in web browsers. On the server-side, technologies like Node.js allow developers to build server applications using JavaScript.
6. *Weakly Typed (Dynamically Typed):* JavaScript is a weakly typed language, meaning that variable data types are not explicitly declared. Instead, JavaScript determines data types at runtime, allowing for flexible and dynamic variable assignments.
7. *Imperative and Declarative*: JavaScript supports both imperative and declarative programming paradigms. It allows developers to specify step-by-step instructions (imperative) while also supporting a more declarative style where developers describe what they want to achieve without specifying how to do it in detail.

8. *Standard Library and Core Elements*: JavaScript includes a standard library of built-in objects like arrays, dates, and math functions, making it easier for developers to perform common tasks. It also provides core language elements such as operators (e.g., +, -, , /), control structures (e.g., if statements, loops), and statements (e.g., variable declarations) for creating functional code.

In summary, JavaScript is a versatile and widely used programming language with characteristics that make it well-suited for web development. Its ability to enhance webpage interactivity, support both client-side and server-side development, and its lightweight nature have contributed to its popularity in the world of programming.

**JavaScript can be utilized in HTML files through two primary methods:**

1. **Internal JS:** JavaScript code can be inserted directly into an HTML file by placing it within the **<script>** tags. These **<script>** tags can be positioned in either the **<head>** or the **<body>** section of the HTML document, depending on the specific needs of the code.
2. **External JS:** Alternatively, JavaScript code can be written in separate files with a ".js" extension. These external JavaScript files can then be linked to an HTML file by including a **<script>** tag inside the **<head>** section of the HTML document, specifying the source file using the "src" attribute.

**Syntax:**

```
<script>
  // JavaScript Code
</script>
```

**Example:**

```
<!DOCTYPE html>
<html lang="en">
<head>
        <title>
                Basic Example to Describe JavaScript
        </title>
</head>
```

```
<body>
        <!-- JavaScript code can be embedded inside
                head section or body section -->
        <script>
                console.log("Welcome to Web Programming");
        </script>
</body>
</html>
```

Output:

Welcome to web programming

**JavaScript is a versatile programming language that finds applications in various domains. Here are detailed explanations of its applications:**

1. *Web Development:* JavaScript is primarily known for enhancing web development. It allows developers to add interactivity and behavior to static websites. With JavaScript frameworks like AngularJS, ReactJS, and VueJS, developers can create dynamic, responsive web applications. These frameworks provide tools and components for building user interfaces, handling data, and managing the flow of web applications.
2. *Web Applications*: As web technologies have advanced, the need for robust web applications has grown. JavaScript plays a pivotal role in creating web applications that offer rich user experiences. Modern web apps, like Google Maps, utilize JavaScript to provide features like interactive maps, real-time updates, and seamless navigation. JavaScript interacts with Application Programming Interfaces (APIs) to access data and services, making web apps more powerful and responsive. Frameworks like Electron (for desktop apps) and React (for web apps) facilitate the development of feature-rich applications.
3. *Server Applications*: JavaScript has extended its capabilities from the client side to the server side with the help of Node.js. Node.js is a runtime environment that allows developers to run JavaScript on the server. It is known for its non-blocking, event-

driven architecture, making it efficient for building scalable server applications. Node.js is widely used for creating web servers, RESTful APIs, real-time applications, and microservices.

4. *Games*: JavaScript is not limited to websites; it is also employed in game development. Combining JavaScript with HTML5 and CSS, developers can create interactive and visually appealing games that run directly in web browsers. JavaScript libraries like EaseJS provide solutions for working with rich graphics, animations, and game physics. This makes JavaScript a popular choice for casual and mobile game development.
5. *Smartwatches*: JavaScript is utilized in smartwatch applications as well. The PebbleJS library offers a framework for developing apps that run on Pebble smartwatches. These applications can interact with smartphone data, sensors, and the internet to provide a wide range of functionalities, from fitness tracking to notifications.
6. *Art:* JavaScript can be a creative tool for artists and designers. It enables them to draw and animate on HTML5 canvases, creating interactive and visually engaging art. The p5.js library, for example, simplifies creative coding with JavaScript by providing functions for graphics, animation, and sound manipulation. Artists can use JavaScript to bring their digital art to life.
7. *Machine Learning:* JavaScript has made inroads into the field of machine learning through libraries like ml5.js. Developers can leverage machine learning models in web applications to perform tasks such as image recognition, natural language processing, and sentiment analysis. ml5.js simplifies the integration of machine learning capabilities into web projects, making it accessible to a broader range of developers.
8. *Mobile Applications*: JavaScript is also used to build mobile applications. With frameworks like React Native, developers can write code in JavaScript and use it to create mobile apps for various operating systems, including iOS and Android. React Native allows developers to build cross-platform apps, reducing the need to write platform-specific code. This "write once, use anywhere" approach streamlines mobile app development and accelerates time-to-market.

In summary, JavaScript's versatility and wide range of libraries and frameworks make it a powerful tool for various applications, from web development and mobile app creation to

game development, machine learning, and creative endeavors. Its ubiquity in modern technology makes it an essential skill for developers in today's digital landscape.

**Limitations of JavaScript:**

1. *Security Risks*: One of the significant limitations of JavaScript is the potential for security risks, particularly in web applications. JavaScript can be used maliciously to execute cross-site scripting (XSS) attacks. In an XSS attack, an attacker injects malicious JavaScript code into a web page, which is then executed in the user's browser. This can lead to the theft of sensitive user data, such as login credentials or personal information. JavaScript's ability to fetch data using AJAX or manipulate tags like `<img>`, `<object>`, or `<script>` makes it susceptible to such attacks. To mitigate these risks, developers need to implement proper security measures, including input validation and sanitization, to prevent XSS vulnerabilities.
2. *Performance:* JavaScript is an interpreted language, which means it doesn't provide the same level of performance as compiled languages like C++ or Java. While modern JavaScript engines have improved performance significantly, complex and computationally intensive programs written in JavaScript may still run slower compared to equivalent programs in compiled languages. However, for most web applications, JavaScript's performance is sufficient, and its performance limitations are not a significant restriction.
3. *Complexity:* JavaScript is a versatile and powerful language, but it can also be complex. To become proficient in JavaScript, programmers must have a deep understanding of various programming concepts, including variables, data types, functions, objects, and asynchronous programming. Additionally, they need to be familiar with both client-side and server-side JavaScript, as well as various libraries and frameworks. Writing advanced scripts and applications in JavaScript requires mastering these concepts, which can be challenging for beginners.
4. *Weak Error Handling and Type Checking Facilities*: JavaScript is considered a weakly typed language, which means developers do not need to specify the data type of a variable explicitly. While this flexibility can be advantageous, it also introduces the risk of type-related errors. JavaScript's weak type checking facilities may not catch type-related errors during compilation, leading to unexpected behavior or bugs at runtime.

Developers need to be vigilant in ensuring that their code handles type-related issues properly to prevent unexpected results.

In summary, JavaScript is a powerful and versatile language widely used for web development and beyond. However, its limitations, such as security risks, performance constraints for complex computations, complexity, and weak error handling and type checking, should be considered by developers when designing and implementing JavaScript applications. By addressing these limitations through best practices and careful coding, developers can harness the full potential of JavaScript while minimizing its drawbacks.

**JavaScript as a Lightweight Language**

JavaScript is often described as a lightweight programming language, primarily due to its minimalistic syntax, low CPU usage, and ease of implementation. One notable feature contributing to its lightweight nature is its lack of strict data types, where everything is treated as an object. This characteristic simplifies the learning curve, as JavaScript shares a syntax similar to languages like C++ and Java.

Being lightweight means that JavaScript places minimal demands on your CPU and system resources. It operates efficiently in web browsers, even when handling complex logic and paradigms. This efficiency translates to reduced resource consumption compared to other languages. For instance, Node.js, a variant of JavaScript, not only delivers faster computations but also operates with fewer resource requirements compared to alternatives like Dart or Java.

However, this lightweight nature also comes with a drawback. JavaScript has relatively fewer built-in libraries and frameworks compared to some other languages. Consequently, developers often need to incorporate external libraries and frameworks to expand its capabilities. Despite this, JavaScript's lightweight design remains a key factor in its popularity and widespread use in web development and beyond.

**Is JavaScript compiled or interpreted or both:**

JavaScript is a language that can be both compiled and interpreted, with its behavior evolving over time. In its earlier versions, JavaScript relied solely on interpretation, where it executed code line by line and provided immediate results. However, as performance concerns emerged due to the slower nature of interpretation, newer versions of JavaScript, particularly after the introduction of V8, incorporated a Just-In-Time (JIT) compiler to optimize execution and deliver faster results.

This JIT compiler functions by generating bytecode, a set of highly optimized instructions that facilitate quicker code execution. Initially, the V8 engine, one of the most prominent JavaScript engines, interprets the code. As the code is executed repeatedly, the V8 engine identifies patterns such as frequently used functions and variables, subsequently compiling them to enhance performance.

While JavaScript is widely recognized for its role in web page development, it has also found applications beyond web browsers, demonstrating its versatility and adaptability in various environments.

INSPIRED BY LIFE

# 3. HISTORY AND VERSIONS OF JAVASCRIPT

## 3.1 History

The history of JavaScript is a fascinating journey that started in the mid-1990s.

1. *Birth of JavaScript:* JavaScript was created by Brendan Eich, who was working as an engineer at Netscape Communications Corporation. In September 1995, he was tasked with developing a scripting language for the Netscape Navigator web browser. The initial project, codenamed "Mocha," aimed to provide a way to make web pages more interactive.
2. *LiveScript to JavaScript:* The language was originally named "LiveScript." However, shortly before its official release, Netscape forged a partnership with Sun Microsystems, which was promoting its own programming language, Java. To leverage the popularity of Java, Netscape decided to rename "LiveScript" to "JavaScript" in December 1995. It was a marketing move to associate the new language with the Java programming language, even though the two languages had significant differences.
3. *First Release:* JavaScript made its debut in Netscape Navigator 2.0 in September 1995. This marked the first time that a scripting language was directly integrated into a web browser. JavaScript allowed web developers to create interactive and dynamic web content, making the web more engaging for users.
4. *ECMAScript Standardization:* As JavaScript gained popularity, there was a need for standardization to ensure compatibility among different web browsers. In 1997, the European Computer Manufacturers Association (ECMA) established a committee to create a standardized specification for JavaScript. The result was the ECMAScript standard, which serves as the basis for JavaScript and its various implementations. The first edition of the ECMAScript standard was released in 1997.
5. *Browser Wars:* During the late 1990s and early 2000s, the "Browser Wars" were in full swing, with Microsoft's Internet Explorer and Netscape Navigator competing for dominance. Each browser had its own implementation of JavaScript, leading to compatibility issues and the need for developers to write code that worked in both browsers.
6. *JavaScript's Resurgence:* JavaScript continued to evolve, and with the release of Internet Explorer 5 in 1999, it gained more capabilities. The introduction of XMLHttpRequest in

Internet Explorer 5 allowed for asynchronous communication with servers, leading to the rise of Ajax (Asynchronous JavaScript and XML) and more dynamic web applications.

7. *JavaScript in the Modern Web*: JavaScript's importance in web development grew exponentially in the 2000s and beyond. The development of powerful JavaScript libraries and frameworks like jQuery, AngularJS, React, and Vue.js simplified web development and made it easier to create complex, interactive web applications. JavaScript is now an integral part of front-end and back-end web development, thanks to technologies like Node.js.
8. *ECMAScript 6 (ES6):* The ECMAScript standard continued to evolve, with ES6 (also known as ECMAScript 2015) being a significant milestone. ES6 introduced many new features and syntax improvements to JavaScript, making it more powerful and developer-friendly.
9. *Modern Browsers and JavaScript Engines:* Modern web browsers, including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge, have highly optimized JavaScript engines that execute code much faster than early implementations. This has enabled the development of complex web applications and online services.
10. *JavaScript Everywhere:* JavaScript is no longer confined to web browsers. It has found its way into various other environments, including server-side development (Node.js), mobile app development (React Native, NativeScript), desktop application development (Electron), and even embedded systems.

In summary, JavaScript's journey from its inception as "LiveScript" to becoming one of the most widely used programming languages in the world has been marked by continuous innovation, standardization efforts, and its adaptation to a wide range of technological advancements. Today, JavaScript is an essential tool for web developers and plays a crucial role in shaping the modern web experience.

## 3.2 Versions

JavaScript, as a programming language, has gone through various versions and iterations over the years. Let's explore the major versions and their key features:

1. *JavaScript (ES1 - ECMAScript 1 - 1997):* This is the first official version of JavaScript. It introduced fundamental language features, including variables, data types, operators,

and control structures. ES1 laid the foundation for JavaScript's growth as a scripting language for web browsers.

2. *ECMAScript 2 (ES2 - 1998):* ES2 focused on making JavaScript more robust and stable. It introduced features like exception handling (try...catch) and more comprehensive error handling mechanisms.
3. *ECMAScript 3 (ES3 - 1999):* ES3 was a significant step forward and became the foundation for JavaScript in web development for many years. It introduced features like regular expressions, better string handling, and support for try...catch...finally for more advanced error handling.
4. *ECMAScript 4 (ES4 - Abandoned):* ES4 was an ambitious attempt to introduce major changes and enhancements to the language. However, it faced strong opposition, and the project was eventually abandoned due to disagreements within the JavaScript community.
5. *ECMAScript 5 (ES5 - 2009):* ES5 brought important improvements to JavaScript, making it more robust and versatile. It introduced features like strict mode, which helped developers write safer and cleaner code. It also added native JSON support and several array manipulation methods (e.g., forEach, map, filter, reduce).
6. *ECMAScript 6 (ES6 - ECMAScript 2015):* ES6 was a game-changer for JavaScript, introducing a wide range of new features and syntax improvements. Some notable additions include:
   - Arrow functions: A concise way to define functions.
   - Class syntax: A more structured and object-oriented way to create constructor functions.
   - Let and const: Block-scoped variables that improve variable handling.
   - Template literals: Enhanced string interpolation.
   - Promises: Improved handling of asynchronous operations.
   - Modules: A standardized way to organize and import/export code.
7. *ECMAScript 2016 (ES7):* ES7 was a smaller update that introduced two significant features:
   - Array.includes(): A method to check if an array includes a specific element.
   - Exponentiation operator: A shorthand for exponentiation (e.g., ).
8. *ECMAScript 2017 (ES8):* ES8 introduced several new features, including:

- Async/await: Simplified asynchronous programming.
- Object.entries(): A method to extract object property keys and values as pairs.
- String padding methods: padStart() and padEnd() for string manipulation.

9. *ECMAScript 2018 (ES9*): ES9 brought some useful additions, such as:
   - Rest/spread properties: Improved object and array handling.
   - Asynchronous iteration: Simplified asynchronous iteration with for-await-of.
   - Promise.prototype.finally(): A method to run code regardless of Promise resolution.
10. *ECMAScript 2019 (ES10):* ES10 added several smaller features, including:
    - Array.flat() and Array.flatMap(): Methods for working with nested arrays.
    - Object.fromEntries(): A method to convert key-value pairs into an object.
    - String.trimStart() and String.trimEnd(): Methods for trimming whitespace.
11. *ECMAScript 2020 (ES11):* ES11 introduced a handful of new features, including:
    - Optional chaining (?.): A concise way to access nested properties without causing errors.
    - Nullish coalescing (??): A way to provide default values for null or undefined variables.
    - Dynamic import(): An asynchronous way to load modules.
12. *ECMAScript 2021 (ES12):* ES12 brought new features like:
    - String.prototype.replaceAll(): A method for replacing all occurrences of a substring.
    - Promise.any(): A method that resolves when at least one promise is resolved.
    - Logical assignment operators (||=, &&=): Simplify assignment based on conditions.

These are the major versions and key features of JavaScript as defined by the ECMAScript standard. JavaScript continues to evolve with yearly updates, providing developers with new tools and capabilities to create modern web applications.

## 4. JAVASCRIPT SYNTAX

JavaScript syntax refers to the set of rules and conventions that govern how JavaScript code should be written and structured. These rules ensure that JavaScript programs are correctly interpreted and executed by web browsers or other JavaScript runtime environments. Understanding JavaScript syntax is essential for writing functional and error-free code. Here are some fundamental aspects of JavaScript syntax:

1. *Statements:* JavaScript programs are composed of statements. Each statement is a line of code that performs a specific action. Statements are often terminated with a semicolon (;), although in some cases, JavaScript can automatically insert semicolons for you.

Example:

```
let x = 10; // Declaration and assignment statement
console.log(x); // Function call statement
```

2. *Variables:* Variables are used to store and manage data values. In JavaScript, you can declare variables using keywords like `var`, `let`, or `const`. Variable names must follow specific naming rules, such as starting with a letter, underscore, or dollar sign, and they are case-sensitive.

Example:

```
let name = "John"; // Variable declaration and assignment
const age = 30;
```

3. *Data Types:* JavaScript supports various data types, including numbers, strings, booleans, objects, arrays, and more. Understanding data types is crucial for working with different kinds of data.

Example:

```
let number = 42; // Number
let text = "Hello, World!"; // String
let isTrue = true; // Boolean
```

4. *Functions*: Functions are reusable blocks of code that can be called with specific arguments. They are defined using the `function` keyword.

Example:

```
function add(x, y) { // Function declaration
  return x + y;
}
let result = add(3, 4); // Function call
```

5. *Conditional Statements:* Conditional statements, like `if`, `else if`, and `else`, are used to control the flow of a program based on specific conditions.

Example:

```
let age = 25;
if (age >= 18) {
  console.log("You are an adult.");
} else {
  console.log("You are not yet an adult.");
}
```

6. *Loops:* Loops, such as `for` and `while`, are used for repeating a block of code multiple times.

Example:

```
for (let i = 0; i < 5; i++) {
  console.log(i); // Prints numbers from 0 to 4
}
```

7. *Objects:* JavaScript uses objects to represent complex data structures. Objects consist of key-value pairs and can be nested.

Example:

```
let person = {
  name: "Alice",
  age: 28,
```

```
    address: {
        street: "123 Main St",
        city: "Cityville"
    }
};
```

8. *Comments:* Comments are used to add explanations and notes to the code. JavaScript supports both single-line (`//`) and multi-line (`/ /`) comments.

Example:

```
// This is a single-line comment
/
This is a multi-line comment
that spans multiple lines.
/
```

These are some of the fundamental elements of JavaScript syntax. Properly following these rules and conventions is essential for writing clean, readable, and maintainable JavaScript code.

# 5. VARIABLES AND DATATYPES

## 5.1 Variables

In JavaScript, variables are used to store and manage data values. There are three ways to declare variables: `var`, `let`, and `const`. Each has its own characteristics and use cases:

1. *`var`:* Variables declared with `var` have function scope or global scope, which means they are accessible throughout the entire function or globally within the script, depending on where they are declared. However, `var` variables are not block-scoped, meaning they are not limited to the block in which they are defined.

   Example:

   ```
   var name = "John";
   ```

   In this case, the variable `name` is accessible both inside and outside of the function or block where it's declared.

2. *`let`:* Variables declared with `let` have block scope, which means they are only accessible within the block of code where they are defined. This provides better control and avoids unintended variable hoisting or reassignment.

   Example:

   ```
   let age = 30;
   ```

   The variable `age` is only accessible within the block in which it is declared.

3. *`const`:* Variables declared with `const` are also block-scoped like `let`, but they have an additional characteristic of being constant. Once a value is assigned to a `const` variable, it cannot be reassigned to a different value.

   Example:

   ```
   const PI = 3.1416;
   ```

   Here, `PI` is a constant with a fixed value, and any attempt to change it later in the code would result in an error.

When choosing between `var`, `let`, and `const`, it's important to consider the scope and mutability requirements of your variables. Use `let` when you need block-scoped variables that can be reassigned, and use `const` when you want to declare constants. Avoid using `var` in modern JavaScript development because of its less predictable scope behavior.

## 5.2 Datatypes

JavaScript is a dynamically typed language, which means that you don't need to explicitly declare the data type of a variable. Instead, JavaScript automatically determines the data type based on the value assigned to the variable. Here are some of the common data types in JavaScript:

1. *Number:* The `Number` data type represents both integer and floating-point numbers. It can be used for mathematical calculations.

   Example:
   var age = 30;
   var price = 9.99;

2. *String:* The `String` data type represents sequences of characters, such as text. Strings are enclosed in single ('') or double ("") quotes.

   Example:
   var name = "John";
   var message = 'Hello, world!';

3. *Boolean:* The `Boolean` data type has two values: `true` and `false`. It is used for logical operations and conditional statements.

   Example:
   var isTrue = true;
   var isFalse = false;

4. *Object:* The `Object` data type is used to create complex data structures. It consists of key-value pairs, where keys are strings (or symbols) and values can be of any data type, including other objects.

Example:

```
var person = { firstName: "John", lastName: "Doe" };
```

5. *Array:* An `Array` is a special type of object used to store ordered collections of values. Each value in an array is called an element and is accessed by an index.

   Example:

   ```
   var fruits = ["apple", "banana", "cherry"];
   ```

6. *Undefined:* The `undefined` data type represents a variable that has been declared but has not been assigned a value.

   Example:

   ```
   var x;
   ```

7. *Null:* The `null` data type represents the intentional absence of any object or value. It is often used to indicate that a variable should have no value.

   Example:

   ```
   var empty = null;
   ```

8. *Symbol:* Symbols are unique and immutable data types introduced in ECMAScript 6 (ES6). They are often used as object property keys to ensure uniqueness.

   Example:

   ```
   var uniqueSymbol = Symbol("description");
   ```

9. *Function:* In JavaScript, functions are first-class citizens, which means they can be assigned to variables and passed as arguments to other functions.

   Example:

   ```
   function add(a, b) {
      return a + b;
   }
   ```

These are the core data types in JavaScript, and they provide the flexibility needed to work with various kinds of data in your programs. JavaScript's dynamic typing allows you to

change the data type of a variable during runtime, which makes it a versatile and expressive language for web development.

## 6. STATEMENTS

In JavaScript, statements are the individual instructions or commands that make up a program. These statements are executed sequentially, one after the other, in the order in which they appear in the code. Each statement performs a specific action or task, and together, they define the behavior and logic of a JavaScript program. Here are some key points to understand about statements in JavaScript:

1. *Basic Syntax:* JavaScript statements typically end with a semicolon (`;`) to mark the end of a statement, although in some cases, JavaScript can automatically insert semicolons through a process called Automatic Semicolon Insertion (ASI).

```
var x = 10;  // Variable assignment statement
console.log("Hello, world!");  // Function call statement
```

2. *Types of Statements:*
   - Variable Declaration: Statements used to declare variables, like `var x = 10;`.
   - Function Calls: Statements that invoke functions or methods, like `console.log("Hello, world!");`.
   - Conditional Statements: Statements that perform conditional execution, like `if`, `else`, and `switch` statements.
   - Loop Statements: Statements used for repetitive execution, like `for`, `while`, and `do...while` loops.
   - Control Flow Statements: Statements that control the flow of your program, like `break`, `continue`, and `return` statements.
   - Exception Handling: Statements for handling exceptions, like `try`, `catch`, and `throw` statements.
   - Object Creation: Statements for creating objects, like `new` operator and object literals.

3. *Whitespace and Formatting:* JavaScript ignores white spaces, line breaks, and indentation when interpreting statements. However, proper formatting and indentation are essential for code readability and maintainability.

```
var name = "John";
var age = 30;
```

4. *Semicolons:* While JavaScript allows you to omit semicolons in certain cases due to Automatic Semicolon Insertion (ASI), it's a best practice to use semicolons consistently to avoid potential issues and make your code more readable.

```
var a = 5;  // Using semicolons for statement separation
var b = 10;
```

5. *Code Blocks:* You can group multiple statements together using code blocks enclosed in curly braces `{}`. Code blocks are often used with control flow statements, functions, and loops.

```
if (condition) {
// Statements within a code block
console.log("Condition is true");
}
```

In summary, statements in JavaScript are the building blocks of a program. They define the actions and logic that the program should follow, and they are executed sequentially to achieve the desired functionality. Proper use of statements, along with good coding practices, is crucial for writing efficient and maintainable JavaScript code.

## 7. OPERATORS

Operators in JavaScript are symbols or keywords that are used to perform operations on values or variables. These operations can include arithmetic calculations, logical comparisons, assignment, and more. JavaScript provides various types of operators to manipulate data and control the flow of your code. Here are the most common types of operators in JavaScript:

1. *Arithmetic Operators:*
   - Addition (`+`): Adds two values.
   - Subtraction (`-`): Subtracts the right operand from the left operand.
   - Multiplication (`*`): Multiplies two values.
   - Division (`/`): Divides the left operand by the right operand.
   - Modulus (`%`): Returns the remainder of the division.

Example :

```
var x = 10;
var y = 5;
var sum = x + y;  // 15
var difference = x - y;  // 5
var product = x * y;  // 50
var quotient = x / y;  // 2
var remainder = x % y;  // 0
```

2. *Assignment Operators:*
   - Assignment (`=`): Assigns a value to a variable.
   - Add and assign (`+=`), subtract and assign (`-=`), multiply and assign (`*=`), etc.: Combine an operation with assignment.

```
var x = 5;
x += 3;  // x is now 8
```

3. *Comparison Operators:*
   - Equal to (`==` or `===`): Compares two values for equality.
   - Not equal to (`!=` or `!==`): Compares two values for inequality.
   - Greater than (`>`), Greater than or equal to (`>=`), Less than (`<`), Less than or equal to (`<=`): Compare numeric values.

Example :

```
var a = 5;
var b = 10;
var isEqual = a === b;  // false
var isNotEqual = a !== b;  // true
var isGreaterThan = a > b;  // false
```

4. *Logical Operators:*
   - Logical AND (`&&`): Returns true if both operands are true.
   - Logical OR (`||`): Returns true if at least one operand is true.
   - Logical NOT (`!`): Returns the opposite boolean value of the operand.

Example :

```
var x = true;
var y = false;

var result1 = x && y;  // false

var result2 = x || y;  // true

var result3 = !x;     // false
```

5. *Unary Operators:*
   - Unary plus (`+`): Converts its operand to a number.
   - Unary minus (`-`): Negates its operand.

Example:

```
var str = "42";
var num1 = +str;   // 42 (number)
var num2 = -str;   // -42 (number)
```

6. *Conditional (Ternary) Operator:*
   - The conditional operator (`? :`) allows you to return a value based on a condition.

Example :

```
var age = 18;
var canVote = age >= 18 ? "Yes" : "No";  // "Yes"
```

7. *Typeof Operator:*
   - The `typeof` operator returns a string indicating the type of a variable or expression.

Example:

```
var x = 10;
var type = typeof x;  // "number"
```

These are some of the fundamental operators in JavaScript. They play a crucial role in performing calculations, making decisions, and controlling the flow of your code. Understanding how to use operators effectively is essential for writing JavaScript programs.

**Console log:**

'console.log ()` is a function in JavaScript that is commonly used for debugging and displaying messages or data in the web browser's console. Here's an explanation of `console.log ()`:

1. *Output to the Console:* `console.log()` is a built-in JavaScript function that allows you to output information to the console, which is a developer tool available in most web browsers. The console is used for debugging and inspecting web pages.
2. *Displaying Messages:* You can pass one or more arguments to `console.log()`. These arguments can be strings (text) or variables containing data. For example, `console.log("Hello, World!")` will display "Hello, World!" in the console.
3. *Multiple Arguments*: You can pass multiple arguments to `console.log()` by separating them with commas. For instance, `console.log("Name:", "John", "Age:", 30)` will display "Name: John Age: 30" in the console.
4. *Variables and Expressions:* You can use `console.log()` to display the values of variables and the results of expressions. For example:

```
var x = 10;
var y = 20;
```

```
console.log("Sum:", x + y);
```

This will display "Sum: 30" in the console.

5. *Formatting:* You can format the output by using placeholders and substitutions. For example:

```
var name = "Alice";
console.log(`Hello, ${name}!`);
```

This uses a template literal and will display "Hello, Alice!" in the console.

6. *Debugging:* Developers often use `console.log()` for debugging purposes. By inserting `console.log()` statements at different points in their code, they can inspect the values of variables and track the flow of their program to identify and fix issues.

7. *Inspecting Objects*: You can also use `console.log()` to inspect the properties and values of JavaScript objects and arrays. It provides a detailed view of complex data structures.
8. *Browser Compatibility:* `console.log()` works in most modern web browsers, making it a valuable tool for web developers to understand how their JavaScript code is behaving.

Remember that when you use `console.log()` in your code, the messages or data you log will only be visible in the browser's console. It won't affect the appearance or functionality of your web page for regular users; it's purely a developer tool.

**Simple JavaScript program that calculates the area of a rectangle :**

```
// Step 1: Define variables to store the dimensions of the rectangle
var length = 10; // Length of the rectangle (in units, e.g., meters)
var width = 5;   // Width of the rectangle (in units)
// Step 2: Calculate the area of the rectangle using the formula: area = length  width
var area = length  width;
// Step 3: Display the result using console.log()
console.log("The area of the rectangle is: " + area + " square units.");
```

Explanation:

1. *Variable Definition:*

- We start by defining two variables, `length` and `width`, to store the dimensions of the rectangle. In this example, we set the length to 10 units and the width to 5 units. You can change these values to calculate the area of different rectangles.

2. *Area Calculation:*
   - We calculate the area of the rectangle using the formula `area = length  width`. This is a straightforward multiplication of the length and width values.
3. *Displaying the Result:*
   - Finally, we use `console.log()` to display the calculated area. The `console.log()` function is used for printing messages to the browser's console. In this case, we concatenate the result with a message to make it more informative.

When you run this JavaScript program, it will output:

```
The area of the rectangle is: 50 square units.
```

This program demonstrates the basic structure of a JavaScript program, including variable declaration, mathematical operations, and outputting results to the console. You can modify the `length` and `width` values to calculate the area of rectangles with different dimensions.

## 8. SUMMARY

The chapter covers various fundamental aspects of the JavaScript programming language. It begins with an introduction to JavaScript, describing it as a lightweight, cross-platform, and versatile language known for web development. JavaScript is both interpreted and compiled, making it suitable for a wide range of applications, including client-side and server-side development. The history and versions of JavaScript are discussed, highlighting its creation in 1995 by Brendan Eich at Netscape. JavaScript's evolution from a simple scripting language to a versatile tool used in web and non-browser environments is explained. The syntax of JavaScript is introduced, emphasizing the set of rules that govern how JavaScript programs are structured. Variables and data types are explored, including numbers, strings, booleans, objects, and arrays. Statements, which are the basic programming instructions in JavaScript, are discussed. The use of semicolons to separate statements and the order of statement execution are explained. The chapter concludes with a discussion of operators in JavaScript, which are used to perform various operations on data, including arithmetic, comparison, and logical operations. Overall, the chapter provides a foundational understanding of the JavaScript language, its history, syntax, data types, statements, and operators, laying the groundwork for more advanced programming concepts and techniques.

INSPIRED BY LIFE

## 9. QUESTIONS

### Self-Assessment Questions

**SELF-ASSESSMENT QUESTIONS – 1**

1. What is JavaScript known for in web development?
2. Who is credited with creating JavaScript, and in which year was it created?
3. Is JavaScript a compiled or interpreted language, or both?
4. Define JavaScript syntax.
5. Name two types of data declarations in JavaScript.
6. What are variables used for in JavaScript?
7. Explain the purpose of semicolons in JavaScript.
8. What is the primary function of operators in JavaScript?
9. Describe the difference between a boolean and a string data type in JavaScript.
10. In JavaScript, what does the console.log() function do?

### Terminal Questions

1. Explain the history and versions of JavaScript.
2. Describe JavaScript syntax with examples.
3. Explain the various data types in JavaScript.
4. Discuss the role of statements in JavaScript and provide examples.
5. Explain operators in JavaScript and provide examples.
6. Describe the console.log() function and its usage in JavaScript.

## 10. ANSWERS

**Self-Assessment Questions**

1. JavaScript is known for adding interactivity and behavior to web pages.
2. JavaScript was created by Brendan Eich while he was an engineer at Netscape in 1995.
3. JavaScript is both compiled and interpreted. It uses an interpreter for execution and a JIT (Just-In-Time) compiler for optimization.
4. JavaScript syntax is a set of rules that defines how JavaScript programs are constructed, including how statements, variables, and expressions are written.
5. JavaScript allows variable declarations using var, let, and const.
6. Variables in JavaScript are used to store and manage data values.
7. Semicolons in JavaScript are used to separate statements and mark the end of a statement.
8. Operators in JavaScript are used to perform operations on variables and values, such as addition, subtraction, or comparison.
9. A boolean data type in JavaScript represents either true or false, while a string data type represents text or characters.
10. The console.log() function in JavaScript is used to print messages or values to the console, which is a debugging tool commonly used for displaying output or debugging information.

**Terminal Questions**

1. Refer section 2
2. Refer section 3
3. Refer section 4
4. Refer section 5
5. Refer section 6
6. Refer section 7